

Solver Specification:

highOrderElectroActivationFoamImplicitJfNK

A high-order, implicit, Jacobian-free Newton–Krylov monodomain solver in OpenFOAM with an Eigen linear-algebra back-end

Pablo

May 12, 2026

Abstract

This report specifies the algorithms implemented in the OpenFOAM solver `highOrderElectroActivationFoamImplicitJfNK`. The solver advances the monodomain equations of cardiac electrophysiology coupled to the ten Tusscher–Noble–Noble–Panfilov (TNNP, 2004) ionic cell model. The diffusion operator is discretised with the cell-centred finite-volume framework of OpenFOAM, optionally upgraded to a Local Radial-Extrapolation (LRE) high-order reconstruction; the ionic source is evaluated either at cell centres or at LRE cell quadrature points; time integration follows a θ -scheme (backward Euler or Crank–Nicolson); the resulting nonlinear coupled system is solved by a Jacobian-Free Newton–Krylov (JFNK) method with Armijo backtracking line search, linear extrapolation initial guess and a physical-bracket clamp on the input to the ionic ODE; the inner linear system is solved by an in-house restarted GMRES with modified Gram–Schmidt orthogonalisation built on top of the Eigen sparse linear algebra library; the per-cell ionic ODE uses OpenFOAM’s adaptive RKF45 integrator with configurable tolerances; activation times are tracked by linear interpolation across the first threshold crossing and benchmark points are sampled by inverse-distance weighting. All algorithmic parameters are exposed through the `constant/spatialIntegrationProperties` and `constant/timeIntegrationProperties` dictionaries, and the companion Python driver `run_convergence.py` generates these dictionaries from a single configuration file. This solver is algorithmically identical to its PETSc-based sibling `highOrderElectroActivationFoamImplicitJfNK_PETSc`; the only difference is the linear-algebra library (Eigen vs. PETSc).

Contents

1	Notation	3
2	Governing equations	3
2.1	Monodomain PDE	3
2.2	Ionic ODE	3
2.3	External stimulus	3
3	Time discretisation: the θ-scheme	4
4	Spatial discretisation	4
4.1	Standard orthogonal FV Laplacian	4
4.2	High-order LRE diffusion	4
4.3	Mass matrices	4
4.4	High-order reconstruction at ionic quadrature points	5
4.5	High-order cell averaging of I_{ion}	5

5	Nonlinear solver	5
5.1	Jacobian-Free Newton–Krylov	5
5.2	In-house restarted GMRES	5
5.3	Armijo backtracking line search	6
5.4	Linear extrapolation initial guess	6
5.5	Protective Vm clamp on ODE input	6
5.6	Convergence criteria	6
5.7	Rollback on non-convergence	6
5.8	Alternative nonlinear methods	6
6	Linear solver: Eigen back-end	7
7	Ionic ODE integration	7
8	TNNP numerical protection	7
9	Activation tracking and benchmarks	8
10	Solver flow chart	8
11	Driver script: run_convergence.py	10
11.1	Geometry and parameter sweep	10
11.2	Nonlinear solver (JFNK / Picard)	10
11.3	JFNK matrix-free options	10
11.4	Linear solver (Eigen)	11
11.5	Inner ODE integrator (RKF45)	11
11.6	LRE high-order parameters per p-level	11
11.7	Post-processing and run-control	11
A	Dimension table	12
B	File map	12
C	Differences vs. the PETSc sibling	12

1 Notation

Symbol	Meaning	Units
$\Omega \subset \mathbb{R}^d$	Spatial domain ($d \in \{2, 3\}$)	m
$\partial\Omega$	Insulated boundary (zero-flux Neumann)	—
$t \in (0, T]$	Time	s
$V_m(\mathbf{x}, t)$	Transmembrane voltage	V
$\mathbf{s}(\mathbf{x}, t) \in \mathbb{R}^{17}$	TNNP ionic states	mixed
$I_{\text{ion}}(V_m, \mathbf{s})$	Total ionic current density	V s^{-1}
$I_{\text{ext}}(\mathbf{x}, t)$	External stimulus current density	A m^{-3}
χ	Membrane surface-to-volume ratio	m^{-1}
C_m	Membrane capacitance per unit area	F m^{-2}
$\mathbf{D}(\mathbf{x})$	Bulk conductivity tensor	S m^{-1}
$\theta \in \{0.5, 1\}$	Time-stepping parameter (CN or BE)	—
Δt	Outer time-step length	s
N_c	Number of mesh cells	—
$\mathbf{V}^n \in \mathbb{R}^{N_c}$	Vector of cell-averaged V_m at time t^n	V
$\mathbf{M}, \mathbf{K}, \mathbf{L}$	Mass, stiffness, scaled diffusion matrix	—
$\mathbf{A}, \mathbf{B}$	Implicit LHS / RHS operators of the θ -scheme	—
$\mathbf{F}(\mathbf{x})$	Discrete nonlinear residual to be zeroed	—

2 Governing equations

2.1 Monodomain PDE

The bulk voltage is propagated by the standard monodomain reaction–diffusion equation [1, 2]:

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - I_{\text{ion}}(V_m, \mathbf{s}) + I_{\text{ext}} \quad \text{in } \Omega \times (0, T], \quad (1)$$

subject to insulated boundaries $\mathbf{n} \cdot (\mathbf{D} \nabla V_m) = 0$ on $\partial\Omega \times (0, T]$, and initial condition $V_m(\mathbf{x}, 0) = -84 \text{ mV}$. The conductivity tensor $\mathbf{D}$ is symmetric positive-definite and either uniform or read as a `volTensorField` from disk.

Dividing (1) by χC_m gives the form actually used inside the solver,

$$\frac{\partial V_m}{\partial t} = \mathcal{L} V_m + s(V_m, \mathbf{s}), \quad \mathcal{L} = \frac{1}{\chi C_m} \nabla \cdot (\mathbf{D} \nabla \cdot), \quad s = -\frac{I_{\text{ion}}}{\chi C_m} + \frac{I_{\text{ext}}}{\chi C_m}. \quad (2)$$

2.2 Ionic ODE

At every spatial sampling point the TNNP state vector $\mathbf{s} \in \mathbb{R}^{17}$ evolves according to

$$\frac{d\mathbf{s}}{dt} = \mathbf{f}_{\text{TNNP}}(V_m, \mathbf{s}; \text{tissue}), \quad \mathbf{s}(0) = \mathbf{s}_0, \quad (3)$$

where `tissue` selects `epicardialCells`, `mCells` or `endocardialCells`. The 17 tracked states are

$$\mathbf{s} = (V, K_i, Na_i, Ca_i, Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g, Ca_{SR}).$$

The total ionic current returned to the PDE is the algebraic `Iion_cm` of TNNP [1].

2.3 External stimulus

The external current is a time- and region-windowed step:

$$I_{\text{ext}}(\mathbf{x}, t) = \begin{cases} I_0 & \mathbf{x} \in \Omega_{\text{stim}}^{(b)}, \quad t \in [t_0^{(b)}, t_0^{(b)} + \tau], \quad b = 1, \dots, B, \\ 0 & \text{otherwise,} \end{cases} \quad (4)$$

with $I_0 = 50\,000 \text{ A m}^{-3}$ and $\tau = 2 \text{ ms}$ as Niederer-benchmark defaults.

3 Time discretisation: the θ -scheme

Time is advanced from t^n to $t^{n+1} = t^n + \Delta t$ by

$$\frac{\mathbf{V}^{n+1} - \mathbf{V}^n}{\Delta t} = \theta \mathbf{L} \mathbf{V}^{n+1} + (1 - \theta) \mathbf{L} \mathbf{V}^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n. \quad (5)$$

implicitScheme	θ	Scheme
backwardEuler	1	First-order, L-stable
crankNicolson	1/2	Second-order, A-stable

After multiplication by Δt^{-1} the implementation forms

$$[\mathbf{M}/\Delta t - \theta \mathbf{L}] \mathbf{V}^{n+1} = [\mathbf{M}/\Delta t + (1 - \theta) \mathbf{L}] \mathbf{V}^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n, \quad (6)$$

with the two Eigen `SparseMatrix<scalar, RowMajor>` operators

$$\mathbf{A} = \mathbf{M}/\Delta t - \theta \mathbf{L}, \quad \mathbf{B} = \mathbf{M}/\Delta t + (1 - \theta) \mathbf{L}. \quad (7)$$

$\mathbf{A}$ and $\mathbf{B}$ are rebuilt at every time step by copying $\mathbf{M}$, scaling by Δt^{-1} and adding $\pm \theta \mathbf{L}$. When $\theta = 1$ (BE) the explicit half of the diffusion vanishes and $\mathbf{B} = \mathbf{M}/\Delta t$. The scaled diffusion is $\mathbf{L} = \mathbf{K}/(\chi C_m)$.

4 Spatial discretisation

4.1 Standard orthogonal FV Laplacian

`assembleStandardOrthogonalStiffnessMatrix` computes the classical 5/7-point Laplacian. Per internal face f with face area $\mathbf{S}_f$,

$$a_f = \frac{|\mathbf{S}_f| (\mathbf{n}_f \cdot \mathbf{D}_f \cdot \hat{\mathbf{d}}_f)}{|\mathbf{d}_f|}, \quad \mathbf{D}_f = \frac{1}{2} (\mathbf{D}_{\text{own}} + \mathbf{D}_{\text{nei}}), \quad (8)$$

contributing $-a_f/V_{\text{own}}$ to $\mathbf{K}[\text{own}, \text{own}]$, $+a_f/V_{\text{own}}$ to $\mathbf{K}[\text{own}, \text{nei}]$, and the transposed pair to the neighbour row. `zeroGradient/empty` patches contribute zero flux. `fixedValue` patches contribute to the owner diagonal only.

4.2 High-order LRE diffusion

When `useHighOrder_Vm = true` and $d > 1$, `assembleHighOrderStiffnessMatrix` integrates the flux on a face quadrature with reconstructed gradients:

$$\Phi_f = \sum_{q \in \mathcal{Q}_f} w_q |\mathbf{S}_f| \mathbf{n}_f \cdot \mathbf{D}_f \cdot \left(\sum_{j \in \mathcal{S}_f} \mathbf{g}_{f,q,j} \mathbf{V}_j \right), \quad (9)$$

where $\mathcal{S}_f$ is the LRE face stencil (~ 15 – 50 cells) and $\mathbf{g}_{f,q,j} = \mathbf{QRGradFaceGPCoeffs}$. The contribution of each $\mathbf{V}_j$ is added to row `own` with weight $+\Phi_{f,j}/V_{\text{own}}$ and to row `nei` with weight $-\Phi_{f,j}/V_{\text{nei}}$, yielding a negative semi-definite Laplacian.

4.3 Mass matrices

Two options:

Lumped (`massMatrix lumped`). `assembleDiagonalMassMatrix` writes $\mathbf{M} = \mathbf{I}$.

Consistent HO (`massMatrix consistent`). `assembleConsistentMassMatrixHO` fills

$$\mathbf{M}[i, j] = \sum_{q \in \mathcal{Q}_i} \frac{w_q}{\sum_{q'} w_{q'}} \phi_j(\mathbf{x}_q), \quad \phi_j(\mathbf{x}_q) = \delta_{ij} + \mathbf{g}_{i,j} \cdot \mathbf{d}_q + \frac{1}{2} \mathbf{H}_{i,j} : (\mathbf{d}_q \otimes \mathbf{d}_q) + \frac{1}{6} \mathbf{T}_{i,j}^{(3)} : (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \quad (10)$$

with $\mathbf{d}_q = \mathbf{x}_q - \mathbf{C}_i$. A safety guard silently downgrades to the lumped form when `useHighOrder_Vm = false` or $d \leq 1$.

4.4 High-order reconstruction at ionic quadrature points

If `useHighOrder_Iion = true`, the ionic ODE is integrated at cell quadrature points. V_m is reconstructed there by the same Taylor expansion as in (10):

$$V_m(\mathbf{x}_q) \approx V_{m,c} + (\nabla V_m)_c \cdot \mathbf{d}_q + \frac{1}{2} \mathbf{H}_c : (\mathbf{d}_q \otimes \mathbf{d}_q) + \frac{1}{6} \mathbf{T}_c^{(3)} : (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \quad (11)$$

implemented in `reconstructVmAtIionIntegrationPoints`.

4.5 High-order cell averaging of I_{ion}

`averageIionIntegrationPointsToCells` contracts the per-QP I_{ion} back to cells:

$$I_{\text{ion},i} = \frac{\sum_{q \in \mathcal{Q}_i} w_q I_{\text{ion},q}}{\sum_{q \in \mathcal{Q}_i} w_q}. \quad (12)$$

5 Nonlinear solver

The fully discrete equation is nonlinear in $\mathbf{V}^{n+1}$. We define

$$\mathbf{F}(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{r}(I_{\text{ion}}(\mathbf{x})), \quad \mathbf{x} = \mathbf{V}^{n+1}, \quad (13)$$

with $\mathbf{r}(I_{\text{ion}}) = \mathbf{B}\mathbf{V}^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n$.

5.1 Jacobian-Free Newton–Krylov

Newton iterates $\mathbf{J}(\mathbf{x}_k) \boldsymbol{\delta}_k = -\mathbf{F}(\mathbf{x}_k)$, with $\mathbf{J}$ never assembled. Each matrix–vector product needed by GMRES is approximated by a forward finite difference [3]:

$$\mathbf{J}(\mathbf{x}_k) \mathbf{v} \approx \frac{\mathbf{F}(\mathbf{x}_k + \varepsilon \mathbf{v}) - \mathbf{F}(\mathbf{x}_k)}{\varepsilon}, \quad \varepsilon = \varepsilon_0 \frac{1 + \|\mathbf{x}_k\|_2}{\|\mathbf{v}\|_2}, \quad (14)$$

with $\varepsilon_0 = \text{jfnkEpsilon}$. The well-known optimisation is $\varepsilon_0 \approx \sqrt{\eta}$ where η is the noise floor of $\mathbf{F}$ (here driven by the RKF45 `relTol`).

5.2 In-house restarted GMRES

`solveGMRES` is an in-house implementation of GMRES(m) with modified Gram–Schmidt orthogonalisation and outer restarts. Each inner iteration:

1. builds one Krylov direction via `matVec`;
2. orthogonalises against the running basis V with MGS, updating the upper-Hessenberg matrix H ;
3. solves the small least-squares problem $\min \|g - H_j y\|$ with Eigen’s column-pivoted Householder QR;
4. estimates the relative residual exactly as $\|g - H_j y\| / \|b\|$ (no extra `matVec`).

After `jfnkMaxKrylovIterations` inner steps the Krylov subspace is discarded and the true residual $r = b - A\mathbf{x}$ is recomputed (one extra `matVec`) before the next restart, repeated up to `jfnkMaxRestarts` times.

5.3 Armijo backtracking line search

The unconditional update $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_0 \boldsymbol{\delta}_k$ is wrapped in Armijo backtracking [4]:

$$\alpha_k = \alpha_0 \cdot 2^{-\ell_k}, \quad \ell_k = \min \left\{ \ell \geq 0 \mid \|\mathbf{F}(\mathbf{x}_k + 2^{-\ell} \alpha_0 \boldsymbol{\delta}_k)\|_2^2 \leq (1 - 2c 2^{-\ell} \alpha_0) \|\mathbf{F}(\mathbf{x}_k)\|_2^2 \right\}, \quad (15)$$

with $c = \text{jfnkArmijoC}$. Backtracking stops on the first success or after `jfnkLineSearchMaxIter` trials; an absolute floor `jfnkLineSearchAlphaMin` below which α is forced and the current trial accepted protects against infinite loops. Disable with `jfnkLineSearch false`.

5.4 Linear extrapolation initial guess

`jfnkInitGuessOrder` selects between

$$\mathbf{x}_0 = \mathbf{V}^n \quad (\text{jfnkInitGuessOrder} = 0), \quad (16)$$

$$\mathbf{x}_0 = \mathbf{V}^n + \frac{\Delta t}{\Delta t_{\text{prev}}} (\mathbf{V}^n - \mathbf{V}^{n-1}) \quad (\text{jfnkInitGuessOrder} = 1, \text{ default}). \quad (17)$$

Each cell is clamped to `[jfnkInitGuessVmMin, jfnkInitGuessVmMax]`. On the first time step no $\mathbf{V}^{n-1}$ exists; the solver falls back to order 0.

5.5 Protective Vm clamp on ODE input

When `jfnkClampODEInput = true` (default), the Vm vector passed to `TNNP::solveODE` inside every call to `evaluateNonlinearFields` is clamped to `[jfnkInitGuessVmMin, jfnkInitGuessVmMax]` *in the working buffer only*: `VmIntegrationPoints` (HO-Ion path) or a local `scalarField` copy (otherwise). The persistent fields are never mutated.

5.6 Convergence criteria

Newton converges when all of the following hold:

$$\begin{aligned} \|\mathbf{F}(\mathbf{x}_k)\|_2 / \|\mathbf{x}_k\|_2 &\leq \text{nonlinearTolerance}, \\ \|\mathbf{V}_k - \mathbf{V}_{k-1}\|_2 / \|\mathbf{V}_k\|_2 &\leq \text{nonlinearVmTolerance}, \\ \|I_{\text{ion},k} - I_{\text{ion},k-1}\|_2 / \|I_{\text{ion},k}\|_2 &\leq \text{nonlinearIionTolerance}, \\ (\text{opt.}) \quad \max_i \|\mathbf{s}_k^{(i)} - \mathbf{s}_{k-1}^{(i)}\|_2 / \|\mathbf{s}_k^{(i)}\|_2 &\leq \text{nonlinearStatesTolerance}, \end{aligned}$$

plus $k+1 \geq \text{minNonlinearIterations}$. The last criterion is gated by `nonlinearRequireStatesConvergence`. The coupled residual $\|\mathbf{F}\|/\|\mathbf{x}\|$ has a noise floor $\sim \eta = \text{relTol}$ (RKF45); tolerances below this floor cannot be achieved.

5.7 Rollback on non-convergence

If Newton exits with the iteration budget reached and the convergence flag still false, `nonlinearAcceptUnconverged` decides:

- **false** (default): restore $V_m, I_{\text{ion}}, \mathbf{s}$ to t^n values, warn, and continue;
- **true**: commit the unconverged iterate.

5.8 Alternative nonlinear methods

Selectable by `nonlinearMethod`:

`picard`

Fixed-point with lagged I_{ion} source.

`diagonalIion`

Picard with the diagonal of $\partial I_{\text{ion}} / \partial V_m$ (one-sided FD step `diagonalIionEpsilon`) folded into $\mathbf{A}$.

6 Linear solver: Eigen back-end

The wrapper `solveSparseSystem` drives either of two Eigen sparse solvers on the explicit operator A (Picard / diagonal paths):

linearSolver key	Eigen type	Type
<code>SparseLU</code>	<code>Eigen::SparseLU</code>	Direct LU
<code>BiCGSTAB</code>	<code>Eigen::BiCGSTAB + IncompleteLUT</code>	Iterative

Both share the `tolerance` and `maxIterations` dictionary keys (the former is ignored by `SparseLU` since it is direct). The matrix-free JFNK branch never instantiates these wrappers; instead it calls the in-house GMRES (Section 5.2) on the FD Jacobian shell.

7 Ionic ODE integration

For each cell or cell QP, the TNNP ODE (3) is advanced by the OpenFOAM adaptive integrator selected via the `solver` key. Default RKF45 [6]. Keys read from `constant/timeIntegrationProperties`:

Key	Meaning	Default
<code>solver</code>	Concrete ODE integrator	RKF45
<code>absTol</code>	Absolute tolerance per state	10^{-9}
<code>relTol</code>	Relative tolerance per state	10^{-6}
<code>maxSteps</code>	Cap on sub-steps per outer call	10^4
<code>initialODEStep</code>	Initial sub-step (ms)	10^{-5}

`relTol` drives the noise floor of the matrix-free Jacobian: $\eta_F \sim \text{relTol}$. Tightening `relTol` buys a lower attainable outer tolerance at the cost of more RKF45 sub-steps per Newton iteration.

8 TNNP numerical protection

The master switch `TNNPNumericalProtection` (default `false`) gates a guard `protectState(state, t, qp, phase)` invoked at every external entry into the TNNP kernel (`calculateCurrent`, `solveODE_start`, `solveODE_end`, `derivatives`).

Key	Type	Default
<code>TNNPNumericalProtection</code>	Switch	<code>false</code>
<code>TNNPCLampGates</code>	Switch	<code>true</code>
<code>TNNPCLampVmForRates</code>	Switch	<code>true</code>
<code>TNNPLogNumericalProtection</code>	Switch	<code>true</code>
<code>TNNPConcentrationFloor</code>	scalar (mM)	10^{-12}
<code>TNNPVMinForRates</code>	scalar (mV)	-200
<code>TNNPVMaxForRates</code>	scalar (mV)	$+100$
<code>TNNPVoltageZeroEps</code>	scalar (mV)	10^{-8}
<code>TNNPMaxProtectionLogEntries</code>	label	10^6

When protection is enabled, the guards applied in order are:

1. **Non-finite V** : `!std::isfinite(V) → reset to  $-86.2$  mV.`
2. **V clamp for rates** (if `TNNPCLampVmForRates`): clamp to `[TNNPVMinForRates, TNNPVMaxForRates]`.
3. **$V = 0$ singularity avoidance**: push V off zero by `TNNPVoltageZeroEps` (sign preserving) when $|V|$ falls below that threshold.

4. **Concentration floor** on K_i, Na_i, Ca_i, Ca_{SR} : non-finite or below-floor entries are reset to `TNNPConcentrationFloor`.
5. **Gate clamp** on $Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g$ (if `TNNPCLampGates`): non-finite to zero, clamp to $[0, 1]$.

Each correction is bucketed by (phase, variable, reason) in an in-memory `ProtectionSummaryEntry` table and emitted to `TNNP_numericalProtection_summary.dat` at exit.

9 Activation tracking and benchmarks

Activation time per cell. First-crossing of V_{thr} in a time step is recorded by linear interpolation:

$$w = \text{clip}\left(\frac{V_{\text{thr}} - V^n}{V^{n+1} - V^n}, 0, 1\right), \quad t_{\text{act}} = t^n + w \Delta t, \quad (18)$$

with a per-cell flag cleared after the first crossing so subsequent re-crossings are ignored.

Early termination on P8. When `stopAfterPointActivation = true`, the cell nearest `stopActivationPoint` (P8 in the Niederer benchmark) is monitored and `effectiveEndTime` is set to $t_{\text{act}} + \text{stopDelayAfterActivation}$ once its $t_{\text{act}} > 0$.

Diagonal sampling. `sampleIDW` performs inverse-distance-weighted interpolation at `nDiagonalSamples` points between `diagonalStart` and `diagonalEnd`, with $k = \text{sampleNeighbours}$, $p = \text{samplePower}$.

10 Solver flow chart

The pseudo-code below summarises the per-time-step flow of the solver and shows the `run.convergence.py` `constants` / `dict keys` that control each branch.

Algorithm 1 High-level time-step loop of `highOrderElectroActivationFoamImplicitJfNK`.

```
1: Initialise PETSc-free Eigen back-end and read dictionaries.
2: Assemble  $M$  (massMatrix),  $K$  (useHighOrderVm),  $L = K/(\chi C_m)$ .
3:  $V^0 \leftarrow -84$  mV (initial condition).
4:  $V^{-1} \leftarrow V^0$ ,  $\Delta t_{\text{prev}} \leftarrow 0$ , hasPrev  $\leftarrow$  false.
5: while  $t^n < \text{END\_TIME}$  do
6:    $\Delta t \leftarrow \text{DT\_VALUES}$  (current sweep value).
7:   Build  $A = M/\Delta t - \theta L$ ,  $B = M/\Delta t + (1 - \theta) L$  (implicitScheme  $\rightarrow \theta$ ).
8:   Apply stimulus to  $I_{\text{ext}}$  (stimulusProtocol).
9:   if jfnkInitGuessOrder  $\geq 1$  and hasPrev then
10:     $V_{\text{guess}} \leftarrow V^n + (\Delta t/\Delta t_{\text{prev}})(V^n - V^{n-1})$ , clamp to  $[\text{jfnkInitGuessVmMin}, \text{jfnkInitGuessVmMax}]$ .
11:   else
12:     $V_{\text{guess}} \leftarrow V^n$ .
13:   end if
14:    $V^{n-1} \leftarrow V^n$ ,  $\Delta t_{\text{prev}} \leftarrow \Delta t$ , hasPrev  $\leftarrow$  true.
15:   if nonlinearMethod = JFNK then
16:      $x \leftarrow V_{\text{guess}}$ .
17:     for  $k = 0, 1, \dots, \text{IMPLICIT\_NONLINEAR\_ITERATIONS}-1$  do
18:        $F_k \leftarrow \text{COMPUTERESIDUAL}(x)$  (advances TNNP via ODE\_REL\_TOL, ODE\_ABS\_TOL, clamps  $V_m$ 
19:       by jfnkClampODEInput).
20:       if all convergence tests pass (nonlinearTolerance, ...) then
21:         break
22:       end if
23:       Build matrix-free Jacobian:  $Jv \approx (F(x + \varepsilon v) - F_k)/\varepsilon$  with  $\varepsilon$  from jfnkEpsilon.
24:       Solve  $J\delta = -F_k$  with in-house GMRES (jfnkMaxKrylovIterations, jfnkLinearTolerance).
25:       if jfnkLineSearch then
26:         Armijo backtracking (jfnkArmijoC, jfnkLineSearchMaxIter, jfnkLineSearchAlphaMin):
27:         find  $\alpha$  satisfying  $\|F(x + \alpha\delta)\|^2 \leq (1 - 2c\alpha)\|F_k\|^2$ .
28:       else
29:          $\alpha \leftarrow \text{nonlinearRelaxation}$ .
30:       end if
31:        $x \leftarrow x + \alpha\delta$ .
32:     end for
33:      $V^{n+1} \leftarrow x$ .
34:   else if nonlinearMethod = picard or diagonalIon then
35:     for  $k = 0, 1, \dots, \text{IMPLICIT\_NONLINEAR\_ITERATIONS}-1$  do
36:       Solve  $AV_* = r(I_{\text{ion}}(x_k))$  with Eigen (linearSolver, tolerance).
37:        $x_{k+1} \leftarrow (1-\omega)x_k + \omega V_*$ ,  $\omega = \text{nonlinearRelaxation}$ .
38:       if convergence tests pass then break
39:     end if
40:   end for
41:    $V^{n+1} \leftarrow x_{k+1}$ .
42: end if
43: if not converged and not nonlinearAcceptUnconverged then
44:   Warn; roll back:  $V^{n+1} \leftarrow V^n$ ,  $I_{\text{ion}}^{n+1} \leftarrow I_{\text{ion}}^n$ ,  $s^{n+1} \leftarrow s^n$ .
45: end if
46: Record activation times (activationThreshold), P8 stop check (stopAfterPointActivation,
47: stopDelayAfterActivation).
48: Write nonlinear-residual log if writeNonlinearResiduals.
49:  $t^{n+1} \leftarrow t^n + \Delta t$ .
50: end while
51: Post-process: IDW sampling at P8 and diagonal (nDiagonalSamples, sampleNeighbours, samplePower).

---


```

Legend. Blue = Python constant set in `run_convergence.py`; green = OpenFOAM dictionary key read by the solver. Constants and keys are mapped one-to-one in section 11.

11 Driver script: `run_convergence.py`

The Python driver generates the OpenFOAM dictionaries and runs the solver in a parameter sweep. Tables below group constants by topic; *Key* is the dictionary entry emitted by the script.

11.1 Geometry and parameter sweep

Python constant	OpenFOAM key	Meaning
DIMENSIONS	—	List of dimensions to sweep (2D, 3D).
DX_VALUES_MM	blockMeshDict cell counts	Uniform cell size in millimetres.
DT_VALUES	controlDict.deltaT	Outer time-step sizes (seconds).
END_TIME	controlDict.endTime	End time (s).
CFL	timeIntegrationProperties.CFL	Informational reference; not enforced.
ACTIVE_TIME_SCHEMES	spatialIntegrationProperties{timeIntegratorScheme	{backwardEuler, crankNicolson}.
ACTIVE_MASS_MATRICES	spatialIntegrationProperties{massMatrixConsistent}	{massMatrixConsistent}.
ACTIVE_HO_PAIRS_Vm_States	useHighOrder_Vm useHighOrder_Iion LRECoeffs_*	/ Tuples (Vm-level, Iion-level); + level $\in \{N0, p1, p2, p3\}$.
ACTIVE_NONLINEAR_METHODS	nonlinearMethod	{JFNK, picard, diagonalIion, ...}.
ACTIVE_STATE_ODE_SOLVERS	timeIntegrationProperties.odeIntegratorNames	ODE integrator names (RK45, ...).
SIGMA_X, SIGMA_Y, SIGMA_Z	cardiacProperties	Diagonal of D .
CHI	cardiacProperties.chi	Surface-to-volume ratio.
CM	cardiacProperties.Cm	Membrane capacitance.

11.2 Nonlinear solver (JFNK / Picard)

Python constant	OpenFOAM key	Meaning
IMPLICIT_NONLINEAR_ITERATIONS	nonlinearIterations	Newton/Picard iteration cap per step.
IMPLICIT_MIN_NONLINEAR_ITERATIONS	minNonlinearIterations	Minimum Newton iterations.
NONLINEAR_TOLERANCE	nonlinearTolerance	Coupled-residual tolerance $\ F\ /\ x\ $.
NONLINEAR_VM_TOLERANCE	nonlinearVmTolerance	Relative L^2 change in V_m .
NONLINEAR_IION_TOLERANCE	nonlinearIionTolerance	Same on I_{ion} .
NONLINEAR_STATES_TOLERANCE	nonlinearStatesTolerance	Max over states.
NONLINEAR_REQUIRE_STATES_CONVERGENCE	nonlinearRequireStatesConvergence	Gate state test.
NONLINEAR_RELAXATION	nonlinearRelaxation	α_0 (Newton damping / Picard mix).

11.3 JFNK matrix-free options

Python constant	OpenFOAM key	Meaning
JFNK_MAX_KRYLOV_ITERATIONS	jfnkMaxKrylovIterations	GMRES restart cycle m .
JFNK_LINEAR_TOLERANCE	jfnkLinearTolerance	GMRES relative tolerance.

Python constant	OpenFOAM key	Meaning
JFNK_EPSILON	jfnkEpsilon	FD perturbation ε_0 . Set near $\sqrt{\eta}$ with $\eta = \text{relTol}$.
DIAGONAL_IION_EPSILON	diagonalIionEpsilon	FD step for diagonal $\partial I_{\text{ion}}/\partial V_m$.

11.4 Linear solver (Eigen)

Python constant	OpenFOAM key	Meaning
IMPLICIT_LINEAR_SOLVER	linearSolver	{SparseLU, BiCGSTAB}. Used by Picard / diagonal branches only; JFNK uses the in-house GMRES on the FD shell.
IMPLICIT_TOLERANCE	tolerance	BiCGSTAB relative tolerance.
IMPLICIT_MAX_ITERATIONS	maxIterations	BiCGSTAB iteration cap.

11.5 Inner ODE integrator (RK45)

Python constant	OpenFOAM key	Meaning
ODE_ABS_TOL	timeIntegrationProperties.absTol	Absolute tolerance per state.
ODE_REL_TOL	timeIntegrationProperties.relTol	Relative tolerance per state; drives the F-noise floor.
ODE_INITIAL_STEP	initialODEStep	Initial sub-step (ms).
ODE_MAX_STEPS	maxSteps	Cap on RK45 sub-steps per outer call.

11.6 LRE high-order parameters per p-level

P_LEVELS_BY_DIM[dim][p] gives, for each (dim, p): N (polynomial order), Nn (stencil neighbours), k (search radius), maxStencilSize (cap). The script writes these into a nested `highOrderCoeffs{LRECoeffs_Vm{...}; LRECoeffs_Iion{...}}` block whenever HO is enabled for either Vm or I_{ion} .

11.7 Post-processing and run-control

Python constant	OpenFOAM key	Meaning
ACTIVATION_THRESHOLD	activationThreshold	V_{thr} (V) for activation-time crossing.
N_DIAGONAL_SAMPLES	nDiagonalSamples	IDW samples along the diagonal.
SAMPLE_NEIGHBOURS	sampleNeighbours	k -NN for IDW.
SAMPLE_POWER	samplePower	Inverse-distance power.
STOP_AFTER_POINT_ACTIVATION	stopAfterPointActivation	Early-stop on P8.
STOP_DELAY_AFTER_ACTIVATION	stopDelayAfterActivation	Delay after P8 activation (s).
WRITE_NONLINEAR_RESIDUALS	writeNonlinearResiduals	Residual log on/off.
SAVE_LOGS_ON_SUCCESS	—	Python-only; keeps logs after a successful run.

A Dimension table

Field	OpenFOAM dimensionSet	SI
Vm	[1, 2, -3, 0, 0, -1, 0]	volt
Iion	dimVoltage/dimTime	V s ⁻¹
externalStimulusCurrent	dimCurrent/dimVolume	A m ⁻³
conductivity	dimCurrent^2 dimTime^3 / (dimMass dimVolume)	S m ⁻¹
chi	dimless/dimLength	m ⁻¹
Cm	dimCurrent dimTime / (dimVoltage dimArea)	F m ⁻²

B File map

Path (relative to cardiacFoam-pc/)	Role
applications/solvers/.../JfNK.C	Main translation unit.
applications/solvers/.../createFields.H	Field initialisation and dictionary reading.
src/ionicModels/TNNP/TNNP.{H,C}	TNNP cell model and protection guards.
src/ionicModels/TNNP/TNNP_2004.H, *Names.H	Generated CellML kernel.
src/ionicModels/ionicModel/ionicModel.{H,C}	Base class with run-time ODE solver selection.
tutorials_electro/.../run_convergence.py	Driver script.
constant/cardiacProperties	χ , C_m , $\mathbf{D}$, ionic model.
constant/timeIntegrationProperties	ODE solver, tolerances, sub-step cap.
constant/spatialIntegrationProperties	Time scheme, HO flags, JFNK / linear / line-search options.
constant/stimulusProtocol	Stimulus regions and timing.

C Differences vs. the PETSc sibling

Algorithmically the two solvers are identical. The only differences are in the linear-algebra back-end and a few user-visible knobs:

Aspect	This solver (Eigen)	PETSc sibling
Sparse matrix container	Eigen::SparseMatrix	PETSc Mat
Sparse direct solver	Eigen::SparseLU	KSPPREONLY+PCLU
Iterative solver (Picard branch)	Eigen::BiCGSTAB+ILUT	KSP+PC (full PETSc catalogue)
JFNK inner Krylov	in-house GMRES	KSPGMRES on MatShell
Preconditioner inside JFNK	none (Krylov-only)	any PETSc PC
Build-time dependency	header-only Eigen3	PETSc + MPI
Parallel run support	serial only	MPI-parallel out of the box

All convergence-related parameters (`nonlinearTolerance`, `jfnkEpsilon`, `jfnkLineSearch*`, `jfnkInitGuess*`, `jfnkClampODEInput`, `nonlinearAcceptUnconverged`, `ODE_*`) have the same names and meanings in both solvers, so the same `run_convergence.py` configuration produces an algorithmically equivalent run with either back-end (modulo the performance and parallel-scaling differences).

References

- [1] K. H. W. J. ten Tusscher, D. Noble, P. J. Noble, A. V. Panfilov, *A model for human ventricular tissue*, *AJP – Heart and Circulatory Physiology* **286** (2004) H1573–H1589.
- [2] S. A. Niederer et al., *Verification of cardiac tissue electrophysiology simulators using an N-version benchmark*, *Prog. Biophys. Mol. Biol.* **104** (2012) 22–48.
- [3] D. A. Knoll, D. E. Keyes, *Jacobian-Free Newton-Krylov methods: a survey of approaches and applications*, *J. Comp. Phys.* **193** (2004) 357–397.
- [4] J. Nocedal, S. J. Wright, *Numerical Optimization*, 2nd edition, Springer, 2006.
- [5] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd edition, SIAM, 2003.
- [6] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II*, Springer, 1996.